

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Mode: In Class

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I K.Madhulika(192524190) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Madhulika

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

ANALYTICAL PROBLEM SOLVING

- 1 Trace the input expression $y = (p + q) * r - s / t$ through all phases of a compiler by identifying the tokens generated during lexical analysis, constructing the syntax tree during parsing, generating intermediate code, applying possible optimizations, and interpreting the relevance of the final target code produced.
- 2 Analyze the tokenization process of the expression `float total = value--++ + ++count-- * 10.5;` by identifying ambiguities in operator recognition, explaining how a LEX-based lexical analyzer processes such patterns, modifying the rules to resolve conflicts, and justifying the correctness of the updated lexical specification.
- 3 Determine the total number of tokens present in a given C program that includes nested loops, multiple variable declarations, arithmetic expressions, and inline comments, while carefully considering how the lexical analyzer ignores comments and whitespace. Analyze each line of the program to identify keywords, identifiers, operators, constants, and punctuation symbols as individual tokens, and compute the final token count based on proper lexical classification.

```
#include <stdio.h>
```

```
int main() {  
    int i, j, sum = 0;    /* initialize variables */  
  
    for(i = 0; i < 3; i++) {  
        for(j = 0; j < 2; j++) {  
            sum = sum + i * j;  
        }  
    }  
  
    printf("%d\n", sum);  
    return 0;  
}
```

- 4 Construct a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression $10+(0+11)^* 0^*1$
- 5 Construct the language for the following Regular Expressions:
 - a) $(\epsilon+a)^*$
 - b) $(a+b)^*aa$
 - c) a^*ba^*
 - d) $(ab+a)^*$
 - e) $(ab)^*+(ba)^*$

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

COI ATI

Given, expression

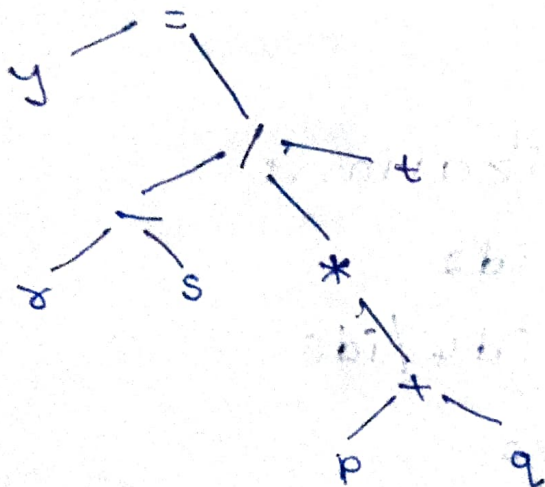
$$y = (p+q) * x - s / t$$

lexemes = $y | = | (| p | + | q | * | x | - | s |$
 $/ | t$

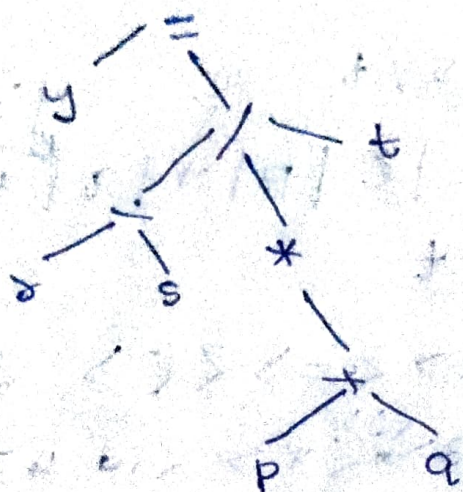
tokens = $\langle id, 1 \rangle \langle = \rangle \langle (\rangle \langle id, 2 \rangle \langle + \rangle$
 $\langle id, 3 \rangle \langle * \rangle \langle id, 4 \rangle \langle - \rangle \langle id, 5 \rangle$
 $\langle / \rangle \langle id, 6 \rangle$

symbol table = y, p, q, x, s, t

syntax tree :



Semantic tree:



intermediate code:

$$t_1 = id_1 + id_2$$

$$t_2 = id_3 - id_4 / id_5$$

$$t_3 = t_1 * t_2$$

$$y = t_3$$

code optimization:

$$t_1 = id_1 + id_2$$

$$t_2 = id_3 - id_4 / id_5$$

$$t_3 = t_1 * t_2$$

$$y = t_3$$

code generation:

MOV R₁, P

MOV R₂, Q

~~MUL~~ R₁, R₂

ADD


```

MOV R3, x
MOV R4, s
MOV R5, t
SUB R3, R4
DIV R3, R5
MUL R1, R3
MOV y, R1

```

Given, expression,

float total = value -- ++ ++ ++ count -- * 10.5;

float keyword 1

total variable 2

= operator 3

value variable 4

-- operator 5

++ operator 6

++ operator 7

++ operator 8

count variable 9

-- operator 10

* operator 11

10.5 constant 12

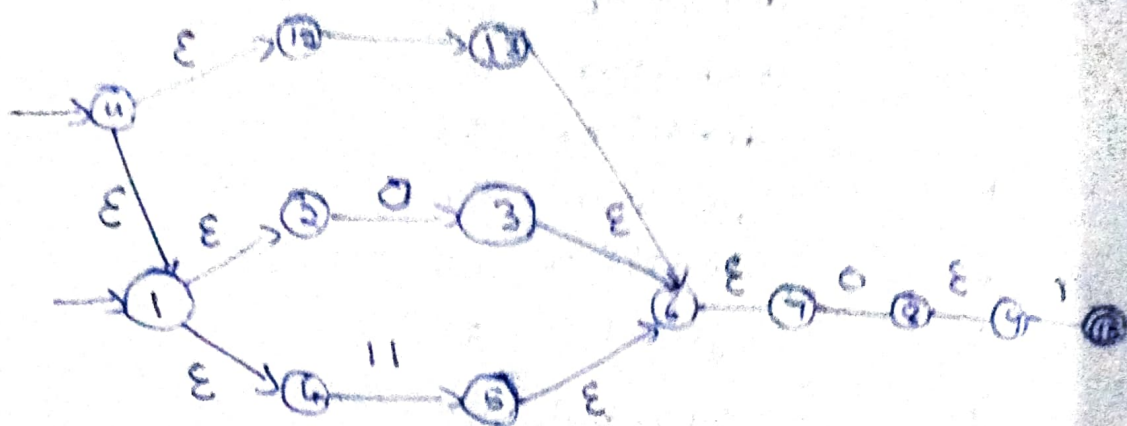
! symbol 13

#	symbol	1	=	symbol	23
include	keyword	2	0	constant	24
<	symbol	3	:	symbol	25
stdio.h	keyword	4	i	identifier	26
>	symbol	5	<	symbol	27
int	keyword	6	3	constant	28
main	keyword	7	;	symbol	29
(	symbol	8	i	identifier	30
)	symbol	9	++	operation	31
{	symbol	10	)	symbol	32
}	symbol	11	{	symbol	33
int	keyword	12	for	keyword	34
i	identifier	13	(	symbol	35
,	symbol	14	)	symbol	36
j	identifier	15	=	operator	37
,	symbol	16	0	constant	38
sum	identifier	17	3	symbol	39
=	operator	18	j	identifier	40
0	constant	19	<	symbol	41
;	symbol	20	2	constant	42
for	keyword	21	:	symbol	43
(	symbol	22	i	identifier	44
i	identifier	23	++	operator	45
			)	symbol	46

{	Symbol	47
sum	identifier	48
=	operator	49
sum	identifier	50
+	operator	51
i	identifier	52
*	operator	53
j	identifier	54
;	symbol	55
}	symbol	56
}	symbol	57
printf	Keyword	58
(	symbol	59
"%d\n"		60
,	symbol	61
sum	identifier	62
)	symbol	63
;	symbol	64
return	Keyword	65
0	constant	66
;	symbol	67
}	symbol	68

4. Given, expression is

$$10 + (0+11)^* 0^*$$



5. a) $(E+a)^*$

$$E+a = \{E, a\}$$

$$L = \{\epsilon, a, aa, aaa, aaaa, \dots\}$$

b) $(a+b)^* aa$

$$L = \{aa, aaa, baa, abaa, bbaa, aabaa, \dots\}$$

c) $a^* ba^*$

$$L = \{b, ba, ab, aba, aab, baa, aaabaa, \dots\}$$

d) $(ab+a)^*$

$$L = \{\epsilon, a, ab, aa, aab, aba, abab, aaba, \dots\}$$

e) $(ab)^* + (ba)^*$

$$L = \{\epsilon, ab, abab, ababab, ba, babab, bababab, \dots\}$$